

▼ Tuning Practice and Comprehensive Problem Solutions

```
# Import libraries
# Data processing, manipulation, and analysis libraries
import numpy as np
import numpy.random as random
import pandas as pd

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

# Machine learning library
import sklearn

# Display up to three decimal places
%precision 3

'%.3f'
```

▼ <Practice Question 1>

Use a different model (e.g., logistic regression) instead of a decision tree and evaluate its performance using k-fold cross-validation.

```
# Answer
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

# Load breast cancer dataset
cancer = load_breast_cancer()

# Initialize Decision Tree model
model = LogisticRegression(random_state=0)

# Perform 5-fold cross-validation
scores = cross_val_score(model, cancer.data, cancer.target, cv=5)

# Display results
print('Cross validation scores:{}'.format(scores))
print('Cross validation scores: {:.2f}+-{:.2f}'.format(scores.mean(), scores.std()))
```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
Cross validation scores:[0.93  0.939 0.965 0.93  0.956]
Cross validation scores:0.94+-0.01
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

✓ <Practice Question 2>

Perform grid search and cross-validation on the breast cancer dataset using a decision tree. For the decision tree parameters, set the tree depth and minimum number of samples per leaf as follows: `param_grid = {'max_depth': [2, 3, 4, 5], 'min_samples_leaf': [2, 3, 4, 5]}`

```
# Answer
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Load the data
cancer = load_breast_cancer()

# Split into training and test data
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify = cancer.target, random_state=0)

# Prepare parameters for GridSearchCV
param_grid = {'max_depth': [2, 3, 4, 5], 'min_samples_leaf': [2, 3, 4, 5]}

# Perform hyperparameter tuning and build the best model
model = DecisionTreeClassifier(random_state=0)
grid_search = GridSearchCV(model, param_grid, cv=5)
grid_search.fit(X_train, y_train)

# Display results
print('Test score:{:.2f}'.format(grid_search.score(X_test, y_test)))
print('Parameters for the best score:{}'.format(grid_search.best_params_))
print('Cross-validation score for the best score:{:.2f}'.format(grid_search.best_score_))

Test score:0.92
Parameters for the best score: {'max_depth': 4, 'min_samples_leaf': 5}
Cross-validation score for the best score:0.94
```

✓ Comments

GridSearchCV:

- Tries all parameter combinations
- Uses 5-fold cross-validation (cv=5)
- Evaluates each combination

For each of the 16 combinations:

- Split training data into 5 folds
- Train on 4 folds, validate on 1
- Repeat 5 times and average

Start coding or [generate](#) with AI.

